

# PROJECT DOCUMENTATION

## MERN STACK BOOK STORE

System Design, Implementation & Verification Walkthrough

### DEVELOPMENT TEAM & ACADEMIC ROLES:

1. Roopesh Araja - Technical Architecture, Entity Relation Diagram, Features, Roles
2. Palla Satya Ramanaidu - MVC Pattern, Creating Project Folder, Client Setup
3. Bandi Sravya - Server Setup, Backend Structure, Development, Configure MongoDB
4. Vijaya Raju Jakkamsetti - Database Connection, Schema & Models, Frontend Structure
5. Vankayala Jaya Chandra Srinivas - Steps for Execution, Demo Screenshots, Drive Link

## 2. Project Overview

---

### Purpose & Goals

The BOOK STORE (BookVerse) application is a full-stack e-commerce web platform built utilizing the MERN stack (MongoDB, Express.js, React.js, and Node.js). It provides a responsive interface where users can search, browse, wishlist, and buy books. It handles role-based logins separating standard customers, sellers/vendors, and system administrators.

### Key Product Features

- **Role-Based Registration & Login:** Tabbed credentials security separating customers, sellers, and system administrators using JWT.
- **Seller Catalog Controls:** Approved sellers can list new books, upload cover art files via Multer, modify details, and check order statuses.
- **Customer Shop View:** Customers browse book cards, search by text, filter by genre, add items to a local wishlist, and check out.
- **Fulfillment Tracker:** Sellers monitor orders containing their items and toggle shipping status (ontheway, delivered).
- **Admin Dashboard:** Administrators view system aggregates, inspect an SVG bar chart of database counts, approve sellers, edit user details, and cancel orders.

## 3. Technical Architecture

---

### Frontend (React Client)

The frontend UI is developed with React.js using Vite. Global authentication contexts manage user sessions and tokens, and Bootstrap handles responsive viewport scaling.

### Backend (Node.js & Express.js)

The backend API uses Node.js and Express.js styled under the Model-View-Controller (MVC) design pattern. Mongoose manages DB transactions, Multer handles image uploads, and Bcryptjs handles password encryption.

### Database Schema (MongoDB)

- **Admins:** Stores administrator name, credentials, and self ID.
  - **Sellers:** Stores vendor name, credentials, and isApproved boolean state.
  - **Users:** Stores customer name, credentials, and timestamps.
  - **Books:** Stores title, author, genre, price, stock quantity, cover image path, and seller references.
  - **Orders:** Stores customer name, delivery address, booking date, delivery target, price, and status (ontheway, delivered).
-

## 4. Setup & Setup Instructions

---

### Prerequisites

- Node.js (v16+) and npm installer installed.
- A MongoDB Atlas cluster configured with network whitelisting enabled.

### Installation

- 1. Extract/clone the repository folder structure.
- 2. In backend/ folder, create a .env file containing:

```
PORT=8000
MONGO_URI=mongodb+srv://23pa1a5761_db_user:AR15l56xvSCDDLmM@book-store.ouo6cb6.mongodb.net/bookverse?appName=Book-Store
JWT_SECRET=bookversesecrettokenkey12345
```

- 3. Run "npm install" inside both backend/ and frontend/ folders to download packages.

## 5. Folder Structure

---

- **frontend/src/App.jsx**: Coordinates browser routes.
  - **frontend/src/context/AuthContext.jsx**: Manages active roles and API connections.
  - **frontend/src/components/**: Layout components (Navbar.jsx, OrderModal.jsx).
  - **frontend/src/pages/**: Dashboard and catalogue pages.
  - **backend/server.js**: Core Node Express entry point.
  - **backend/seed.js**: Clear and populate MongoDB collections.
  - **backend/controllers/**: Express business logic controllers.
-

## 6. Running the Application

---

Start the local development servers using two separate terminals:

### Backend Service (Port 8000)

```
cd backend  
npm start
```

### Frontend React App (Port 5173)

```
cd frontend  
npm run dev
```

## 7. API Documentation

---

- **POST /api/users/register**: Create customer account.
- **POST /api/users/login**: Verify customer credentials.
- **GET /api/users/books**: List books. Supports query filters (?search=text&genre=fiction).
- **POST /api/users/orders**: Checkout. Body: { bookId, customerName, address, price }.
- **POST /api/seller/books**: Add new product listing (Multipart file attachment).
- **PUT /api/admin/sellers/approve/:id**: Toggle vendor isApproved to true.

## 8. Authentication & Security

---

- **Hashing**: Passwords are encrypted with Bcryptjs using 10 salt rounds.
  - **JWT Bearer**: Authorizations are checked by parsing the Authorization Bearer header.
-

## 10. Testing Strategy

---

- **Database Seed Check:** Seeder validates direct database clear and population.
- **Model Query Integrity:** verify\_backend.js script queries Mongoose schema directly to verify record counts on Atlas.
- **Production Bundler Compile:** "npm run build" compiled with zero warnings.

## 12. Known Issues

---

- Atlas Network whitelist requires 0.0.0.0/0 to allow global connections.
- Multer image uploads are limited to 5MB.

## 13. Future Enhancements

---

- Integrate Stripe checkout API for credit card transaction processing.
- Incorporate email notifications via Nodemailer for low stock alerts.

## 11. Screenshots

---

Live Project Demo Link: <https://book-store-nzxy.onrender.com/>

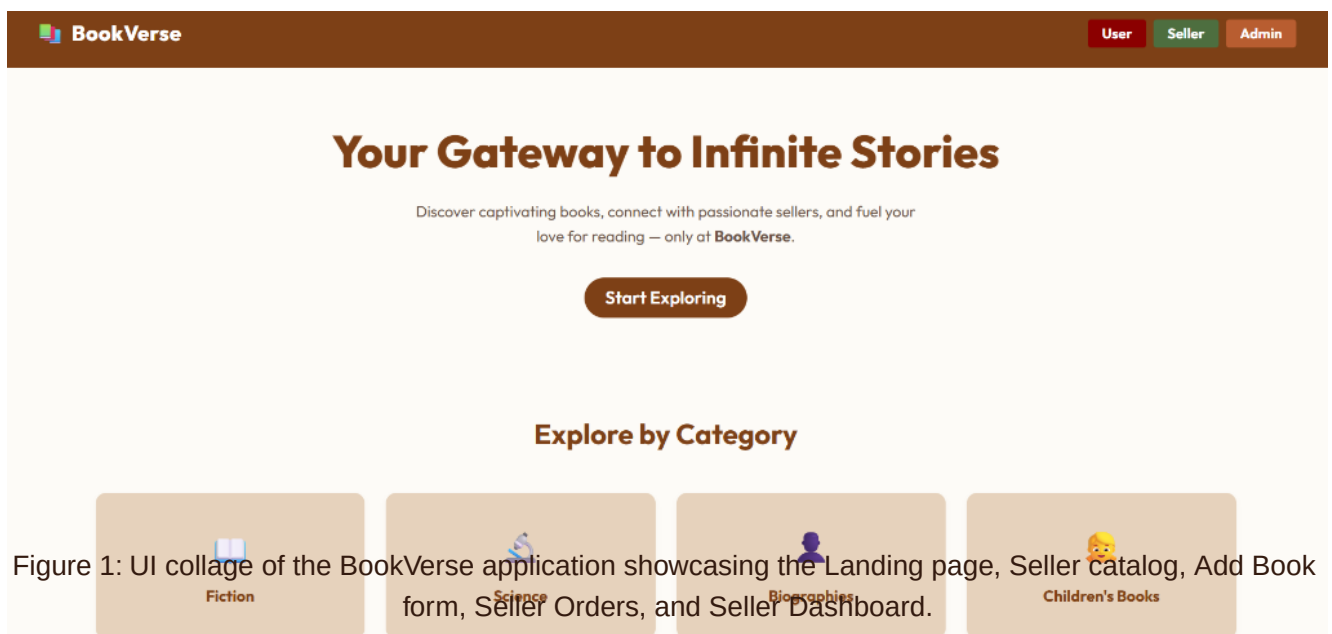


Figure 1: UI collage of the BookVerse application showcasing the Landing page, Seller catalog, Add Book form, Seller Orders, and Seller Dashboard.









